DICOM Header Auto Repair

Maintenance & Deployment Guide

For Medical Physicists and System Administrators

Table of Contents

1. Overview

This document provides comprehensive guidance for medical physicists and system administrators responsible for maintaining, deploying, and extending the DICOM Header Auto Repair tool. The tool automates the detection and correction of common DICOM header errors that prevent successful import into medical imaging systems such as Varian Eclipse.

2. Prerequisites

Knowledge Requirements

- **DICOM Fundamentals:** Understanding of DICOM file structure, tags, and metadata
- **Python Basics:** Familiarity with Python syntax, modules, and functions
- **Command Line:** Basic comfort with Windows PowerShell or Command Prompt
- **File Systems:** Understanding of directory structures and file permissions

Software Requirements

- Python 3.7 or later (download from `python.org`)
- Git (optional, for version control)
- MicroDicom free DICOM viewer[1], for investigating header issues
- 7-Zip (required for creating deployment zip packages)
- Text editor or IDE such as Visual Studio Code

3. System Requirements

Deployment Target Systems

- Windows 7 or later
- Python 3.7 or later installed and available in PATH
- Minimum 500 MB free disk space for DICOM processing
- Read/write access to source and destination directories
- Administrative access for installation (optional, depending on deployment location)

Development Environment

- Windows 10 or later recommended
- Python development tools (IDE or code editor)
- Git for version control (recommended)
- Required Python packages: pydicom and FreeSimpleGUI (see Installation section)

4. Maintenance

Package Structure

Understanding the package structure is essential for maintenance and extension:

```
DicomHeaderAutoRepair/
                ├── dicom_repair.py # Main GUI application entry point
                ├── dicom_repair_gui.py # GUI components and user
interface
                ├── dicom_repair_functions.py # Repair functions
(customize here)
                ├── dicom_repair_tools.py # Utility functions for file
processing
                ├── app_config.py # Application configuration
                ├── dicom_repair_config.toml # Configuration file
                ├── requirements.txt # Python package dependencies
                ├── ReadMe.md # User quick-start guide
                ├── DICOM Test Data/ # Sample DICOM files for testing
                ├── Documentation/ # Additional documentation
                ├── Icons/ # Application icons
                └── install/ # Deployment files
```

Key Components

dicom_repair.py

The main entry point for the application. Launches the GUI and coordinates the repair workflow.

dicom_repair_gui.py

Contains all graphical user interface components. Handles user input, directory selection, and status display.

dicom_repair_functions.py

This is the file you will modify to add custom repair functions. Contains the repair logic for each issue type. Each repair function follows a standard interface and is registered in the `REPAIR_METHODS` list.

dicom_repair_tools.py

Contains utility functions used by repair functions and the main application:

- `perform_repairs()` - Main function that orchestrates file processing
- `count_repairs()` - Summarizes repair statistics
- Other helper functions for DICOM file handling

Dependencies

The package has two primary dependencies:

- **pydicom:** Used for reading and manipulating DICOM file headers and metadata
- **FreeSimpleGUI:** Used to build the graphical user interface (GUI)

Check the current versions in `requirements.txt` :

To update dependencies, edit `requirements.txt` and reinstall:

```
.venv\Scripts\pip.exe install -r requirements.txt
```

Use the virtual environment pip (or activate `.venv` first) so packages are installed into the application environment, not system Python.

Version Control

Maintain version history by tracking changes to key files:

- `dicom_repair_functions.py` - When repair functions are added or modified
- `requirements.txt` - When dependencies are updated
- `app_config.py` - When configuration changes are made

5. Deployment

Deployment Workflow Overview

The deployment process consists of three phases:

1. Run `distribute.bat` on the development machine to create a deployment package

2. Copy the resulting zip file to the target PC
3. Extract the zip file on the target PC and create shortcuts

Pre-Deployment Checklist

- ✓ Test all repairs on sample DICOM files
- ✓ Verify Python version compatibility (3.7+)
- ✓ Update version number and changelog
- ✓ Test on target Windows system
- ✓ All required files are in the project directory
- ✓ Verify 7-Zip is installed at C:\Program Files\7-Zip\7z.exe

Building the Distribution Package

On the development machine, build the distribution package:

Step 1: Prepare the Development Environment

Ensure all repairs and changes are complete and tested. The virtual environment (`.venv`) should be current with all required dependencies.

Step 2: Run the Distribution Script

Navigate to the `install` directory and run `distribute.bat` :

```
cd install distribute.bat
```

This batch file will:

- Copy all Python source files to the `For Distribution` folder
- Copy the configuration file and license
- Copy the application icon
- Copy the entire virtual environment (`.venv` folder) with all dependencies already installed
- Create a zip archive: `For Distribution.zip`

Step 3: Verify the Package

Check that `For Distribution.zip` was created successfully. This file contains everything needed for deployment.

Installation on Target System

Step 1: Transfer the Package

Copy the `For Distribution.zip` file to the target PC using any file transfer method (USB, network share, email, etc.).

Step 2: Extract the Package

On the target PC, extract the zip file to the desired location, typically:

```
C:\Program Files\DicomHeaderAutoRepair
```

Use Windows Explorer or 7-Zip to extract the archive. The directory structure will be created automatically.

Step 3: Verify Installation

Test that the tool launches correctly by double-clicking `DICOM Repair.bat` in the extracted folder. The GUI should appear without errors.

Step 4: Create Shortcuts (Optional)

For user convenience, create a desktop shortcut pointing to `DICOM Repair.bat`. You can:

- Right-click on `DICOM Repair.bat` and select "Create shortcut"
- Move the shortcut to the desktop
- Alternatively, use the existing `DICOM Repair.bat - Shortcut.lnk` file and update its target path

Updating an Existing Installation

To deploy an update to a system that already has the tool installed:

1. Back up the current installation directory
2. Build a new distribution package on the development machine using `distribute.bat`

3. Transfer the new `For Distribution.zip` to the target PC
4. Extract the new package, overwriting the old installation
5. Test the updated version with sample DICOM files
6. Document the update (version, changes, date)

> **Note:** The zip file already contains the complete virtual environment with all dependencies. You do **not** need to run `MakeVenv.bat` after extracting the package unless you need to update the Python environment (e.g., to a newer Python version or updated pydicom library).

When to Use MakeVenv.bat

Use `MakeVenv.bat` only in these scenarios:

- **Environment Update Required:** Python version needs to be upgraded
- **Dependency Update:** A new version of pydicom or FreeSimpleGUI is required
- **Corrupted Environment:** The virtual environment has been corrupted and needs to be rebuilt

If you need to update the environment:

1. Delete the `.venv` folder in the installation directory
2. Run `MakeVenv.bat` from the installation directory
3. Wait for the process to complete (may take several minutes)

> **Tip:** Keep backups of at least the two most recent distribution packages for rollback capability.

6. Extending the Tool with Custom Repairs

Overview

The tool is designed to be extended with new repair functions. When you encounter a DICOM import issue that isn't currently handled, you can add a custom repair function to

automatically fix it.

Anatomy of a Repair Function

Step-by-Step: Creating a New Repair Function

Step 1: Investigate the Problem

Use MicroDicom (see Appendix A) to identify:

- Which DICOM tags are affected
- What values cause the problem

Step 2: Design the Repair Logic

Plan your approach:

- Establish a way to programmatically detect the problem
- Identify the DICOM tags and values that need to be modified
- Determine what the correct value should be (or if the tag should just be removed)
- Consider any edge cases that might need to be handled

Step 3: Write the Repair Function

All repair functions follow the same pattern. Open `dicom_repair_functions.py` and add your function. Use this template:

```
def fix_problem_name(
    dataset: pydicom.Dataset,
    status_output: Callable,
    ) -> pydicom.Dataset:
    """Brief description of what this repair does.

    Detailed explanation of:
    - The problem being fixed
    - DICOM tags involved
    - How the fix is applied

    Args:
        dataset: The full DICOM dataset.
        status_output: Function for reporting results.
```

```
    Returns:
        The modified dataset.
    """
    if "TagName" in dataset:
        value = dataset.data_element("TagName").value

        if problem_detected(value):
            dataset.data_element("TagName").value = fixed_value
            message = f"Problem fixed: {tag_name} changed from X to Y"
            status_output(message)

    return dataset
```

> **Important Note — pydicom.Dataset Tree Structure:** The pydicom.Dataset object has a hierarchical tree structure. Some DICOM tags are nested within sub-sequences rather than at the top level of the dataset. If a tag is not found using the standard `if "TagName" in dataset:` approach, you may need to iterate through the relevant sub-sequences to locate it. See the Example section below for an illustration of iterating through a sequence to access nested elements.

Step 5: Test Your Function

Unit Testing

Create test DICOM files with the problem condition and verify:

- The problem is correctly detected
- The repair is correctly applied
- The output message is informative
- Edge cases are handled

Integration Testing

Run the full tool on test DICOM files:

1. Launch the GUI
2. Select directory with test files

3. Run the repair
4. Verify output files have been corrected
5. Verify status messages appear in the output panel

Validation Testing

Verify the repaired files import successfully:

- Import repaired files into Eclipse
- Verify no import errors
- Confirm file opens without crashing

Best Practices

Error Handling

Always check if tags exist before accessing them:

```
if "TagName" in dataset:
    value = dataset.data_element("TagName").value
else:
    pass
```

Type Safety

Check data types when needed:

```
if isinstance(value, str) and value.startswith("/"):
            pass
```

Logging

Always report meaningful messages:

```
message = (
    f"Problem detected: {tag_name}. "
    f"Value changed from '{old_value}' to '{new_value}'"
)
status_output(message)
```

Documentation

Include detailed docstrings explaining:

- What problem is being fixed
- Which DICOM tags are involved
- Frequency and impact of the problem
- How the fix is applied

Example: Creating a New Repair Function

Scenario: You discover that some PET images have the "Radioactive Administered Activity Units" tag (0018, 1074) in the wrong format, causing import failures.

```
def fix_activity_units(
    dataset: pydicom.Dataset,
    status_output: Callable,
) -> pydicom.Dataset:
    """Fix incorrect activity units in PET images.

    Some PET systems record activity in non-standard units.
    This repair updates values to the expected DICOM standard.
    """
    modality = dataset.get("Modality", None)
    if modality != "PT":
        return dataset

    sequence = dataset.get("RadiopharmaceuticalInformationSequence", [])
    for item in sequence:
        code_sequence = item.get("RadionuclideCodeSequence", [])
        for code_item in code_sequence:
            value = code_item.get("CodeValue", None)
            if value == "WRONG_CODE":
                code_item.CodeValue = "CORRECT_CODE"
                status_output("PET activity units corrected")

    return dataset
```

Then register it in `REPAIR_METHODS`. The next time the tool runs, this repair will be automatically applied to all DICOM files.

Appendix A: Manual Investigation of DICOM Headers

This appendix provides guidance for manually investigating DICOM headers when issues arise. Using tools like MicroDicom, you can examine the raw DICOM metadata to understand what values are present and identify problems that may require repair.

Analyzing Images Using MicroDicom

MicroDicom is a free DICOM viewer[1] that allows you to inspect the header information of DICOM files in detail. Use the following workflow:

Step 1

Open File Explorer in the folder above the folder containing the DICOM images for analysis.

Step 2

Right-click on the folder containing the DICOM images and select "Open with MicroDicom DICOM Viewer".

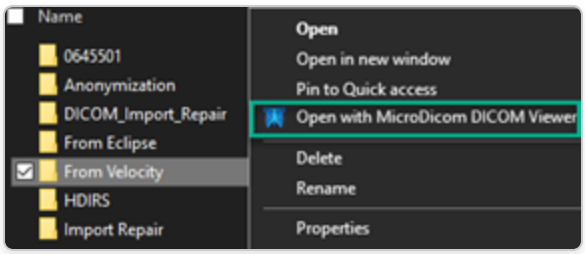

Figure A1: Open Folder with MicroDicom DICOM Viewer

Step 3

The left panel contains the patient information and tree of images. Select the appropriate image set and then any image from within that set.

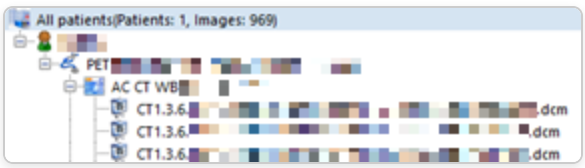

Figure A2: Select Image Set from the Left-Side Tree

Step 4

The right-hand panel displays the DICOM header information as a list of tags.

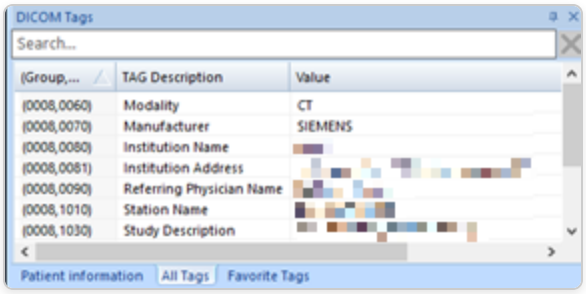

Figure A3: DICOM Tag List

Step 5

If the DICOM tags are not showing, click: View -> DICOM Tags Pane.

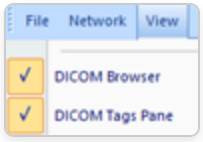

Figure A4: Enable DICOM Tags Pane

Step 6

Import errors can be diagnosed through the contents of the DICOM tags. Use the "PET Import Problems" tables for examples of issues related to DICOM tag information.

Next Steps

Once you have documented the problem:

1. Consult the "Extending the Tool with Custom Repairs" section (Section 6) to create an automated repair
2. Create an anonymized test dataset to confirm that your new repair function works as expected
3. Deploy the updated tool using the steps described in the "Deployment" section (Section 5)

DICOM Header Auto Repair - Maintenance & Deployment Guide

For technical support or questions, contact your system administrator or the development team.

[1] MicroDicom is free for non-commercial use only. Commercial use licenses can be purchased at https://www.microdicom.com/online-store.html.